



Command-line interface (CLI)

The CLI [?](#) command-line interface offers several functionalities related to Odoo. You can use it to [run the server](#), [launch Odoo as a Python console environment](#), [scaffold an Odoo module](#), [populate a database](#) or [count the number of lines of code](#).

Important

The command to use to call the CLI depends on how you installed Odoo. In the examples below, we assume that you are [running Odoo from source](#) with the `odoo-bin` file. If you installed Odoo [from a distribution package](#) or with [Docker](#), you must adapt the command.

Run Odoo from source	Odoo was installed from a distribution package	Odoo was installed with Docker
1. Navigate to the root of the directory where you downloaded the source files of Odoo Community. 2. Run all CLI commands with <code>./odoo-bin</code>		

Version

`-h , --help`

it can be used in combination with any command available, and it displays the options of the current command.

If no command is used, it will act as per the `help` command [below](#).

`--version`

shows Odoo version e.g. "Odoo Server 19.0"

Tip

You can enable auto-completion in your shell by running

help - Show available commands

This command shows all the available commands for Odoo.

It has no options.

server - Run the Server

This command is the default one: you can omit it, and it will be chosen anyway.

-d <database> , --database <database>

database(s) used when installing or updating modules. Providing a comma-separated list restrict access to databases provided in list.

For advanced database options, take a look [below](#).

-i <modules> , --init <modules>

comma-separated list of modules to install before running the server (requires **-d**).

-u <modules> , --update <modules>

comma-separated list of modules to update before running the server. Use `all` for all modules. (requires **-d**).

--reinit <modules>

A comma-separated list of modules to reinitialize before starting the server. (requires **-d**).

The reinitialization is similar to a simple upgrade without executing any upgrade script. It loads data in `init` mode instead of `update` mode, primarily affecting records marked as `'noupdate'` . All modules that depend directly or indirectly on the specified ones will also be reinitialized.

This option is intended for debugging or development purposes only. **Do not use it with a production database.**

--addons-path <directories>

comma-separated list of directories in which modules are stored. These directories are scanned for modules.

--upgrade-path <upgrade_path>

module when an upgrade of any module is requested. This is useful to perform some actions during custom modules upgrade after a major upgrade.

--load <modules>

list of server-wide modules to load. Those modules are supposed to provide features not necessarily tied to a particular database. This is in contrast to modules that are always bound to a specific database when they are installed (i.e. the majority of Odoo addons). Default is `base,web`.

-c <config> , --config <config>

path to an alternate [configuration file](#). If not defined, Odoo checks `ODOO_RC` environmental variable and default location `$HOME/.odoorc`. See configuration file section [below](#).

-D <data-dir-path> , --data-dir <data-dir-path>

directory path where to store Odoo data (eg. filestore, sessions). If not specified, Odoo will fallback to a predefined path. On Unix systems its one defined in `$XDG_DATA_HOME` environmental variable or `~/local/share/Odoo` or `/var/lib/Odoo`.

-s , --save

saves the server configuration to the current configuration file (`$HOME/.odoorc` by default, and can be overridden using `-c`).

--with-demo

install demo data in new databases.

--without-demo

don't install demo data nor in new databases nor when installing new modules in a database that uses demo data, this is the default.

--skip-auto-install

skips auto-installing modules when a new module installation is requested. This option is useful for development. It serves to check if the installed modules do not indirectly depend on auto-installed modules.

--pidfile =<pidfile>

path to a file where the server pid will be stored

--stop-after-init

stops the server after its initialization.

Absolute path to the GeoIP Country database file.

Testing

--test-enable

runs tests after module installation

--test-file <file>

runs a python test file

--test-tags [-][tag][/module][:class][.method]

Comma-separated list of specs to filter which tests to execute. Enable unit tests if set.

Example: `--test-tags :TestClass.test_func,/test_module,external`

- The `-` specifies if we want to include or exclude tests matching this spec.
- The tag will match tags added on a class with a `tagged()` decorator (all **test classes** have `standard` and `at_install` tags until explicitly removed, see the decorator documentation).
- `*` will match all tags.
- If tag is omitted on include mode, its value is `standard`.
- If tag is omitted on exclude mode, its value is `*`.
- The module, class, and method will respectively match the module name, test class name and test method name.

Filtering and executing the tests happens twice: right after each module installation/update and at the end of the modules loading. At each stage tests are filtered by `--test-tags` specs and additionally by dynamic specs `at_install` and `post_install` correspondingly.

--screenshots

Specify directory where to write screenshots when an `HttpCase.browser_js` test fails. It defaults to `/tmp/odoo_tests/db_name/screenshots`

--screencasts

Enable screencasts and specify directory where to write screencasts files. The `ffmpeg` utility needs to be installed to encode frames into a video file. Otherwise frames will be kept instead of the video file.

Database

-r <user> , --db_user <user>

--db_host <hostname>

host for the database server

- localhost on Windows
- UNIX socket otherwise

--db_port <port>

port the database listens on, defaults to 5432

--db_replica_host <hostname>

host for the replica database server, disabled when not set / empty

--db_replica_port <port>

the port the replica database listens on, defaults to **--db_port**

--db-filter <filter>

hides databases that do not match <filter> for the Web UI. The filter is a **regular expression**[↗](#), with the additions that:

- %h is replaced by the whole hostname the request is made on.
- %d is replaced by the subdomain the request is made on, with the exception of www (so domain odoo.com and www.odoo.com both match the database odoo).

These operations are case sensitive. Add option (?i) to match all databases (so domain odoo.com using (?i)%d matches the database 0doo).

Since version 11, it's also possible to restrict access to a given database listen by using the - database parameter and specifying a comma-separated list of databases

When combining the two parameters, db-filter supersedes the comma-separated database list for restricting database list, while the comma-separated list is used for performing requested operations like upgrade of modules.

```
$ odoo-bin --db-filter ^11.*$
```

Restrict access to databases whose name starts with 11

Restrict access to only two databases, 11firstdatabase and 11seconddatabase

```
$ odoo-bin --database 11firstdatabase,11seconddatabase -u base
```

Restrict access to only two databases, 11firstdatabase and 11seconddatabase, and update base module on one database: 11firstdatabase. If database 11seconddatabase doesn't exist, the database is created and base modules is installed

```
$ odoo-bin --db-filter ^11.*$ --database 11firstdatabase,11seconddatabase -u base
```

Restrict access to databases whose name starts with 11, and update base module on one database: 11firstdatabase. If database 11seconddatabase doesn't exist, the database is created and base modules is installed

Warning

This option does not affect cron workers, if no `--database` is given, cron workers will run on every available database

--db-template <template>

when creating new databases from the database-management screens, use the specified [template database](#). Defaults to `template0`.

--pg_path </path/to/postgresql/binaries>

Path to the PostgreSQL binaries that are used by the database manager to dump and restore databases. You have to specify this option only if these binaries are located in a non-standard directory.

--no-database-list

Suppresses the ability to list databases available on the system

--db_sslmode

Control the SSL security of the connection between Odoo and PostgreSQL. Value should be one of 'disable', 'allow', 'prefer', 'require', 'verify-ca' or 'verify-full' Default value is 'prefer'

--unaccent

Try to enable the unaccent extension when creating new databases

--from-filter <address or domain>

Define which email address the SMTP configuration will apply to. The field can be a domain name or an entire email address, or it can remain empty. If the sender's email address does not match this set filter, then the email will be encapsulated using a combination of the two system parameters: `mail.default.from` and `mail.catchall.domain`. For example, "Admin" <admin@example.com> ⇒ "Admin" <notifications@mycompany.com>.

--smtp <server>

Address of the SMTP server to connect to in order to send mails

--smtp-port <port>**--smtp-ssl**

If set, odoo should use SSL/STARTSSL SMTP connections

--smtp-user <name>

Username to connect to the SMTP server

--smtp-password <password>

Password to connect to the SMTP server

--smtp-ssl-certificate-filename <path/to/cert.pem>

An SSL certificate is to be used for authentication. If set, then `smtp-ssl-private-key` is required.

--smtp-ssl-private-key-filename <path/to/key.pem>

An SSL private key is used for authentication. If set, then `smtp-ssl-certificate` is required.

Internationalisation

--load-language <languages>

specifies the languages (separated by commas) for the translations you want to be loaded

--i18n-overwrite

overwrites existing translation terms on updating a module or importing a CSV or a PO file.

Advanced Options

Developer features

--dev <feature, feature, ..., feature>

- `xml` : read QWeb template from xml file directly instead of database. Once a template has been modified in database, it will be not be read from the xml file until the next update/init. Particularly, templates are not translated on using this option.
- `reload` : restart server when python file are updated (may not be detected depending on the text editor used)
- `qweb` : break in the evaluation of QWeb template when a node contains
`t-debug='debugger'`
- `werkzeug` : display the full traceback on the frontend page in case of exception
- `replica` : simulate `--db_replica_host` but connect to the same database server as `--db_host` , this makes it possible to test read-only features without the need to set up a replicated database.
- `access` : log the traceback next to the `AccessError` when it results in a 403 - Forbidden HTTP response.

HTTP

`--no-http`

do not start the HTTP or long-polling workers (may still start `cron` workers)

Warning

has no effect if `--test-enable` is set, as tests require an accessible HTTP server

`--http-interface <interface>`

TCP/IP address on which the HTTP server listens, defaults to `0.0.0.0` (all addresses)

`-p <port>`

`--http-port <port>`

Port on which the HTTP server listens, defaults to 8069.

`--gevent-port <port>`

TCP port for websocket connections in multiprocessing or gevent mode, defaults to 8072. Not used in default (threaded) mode.

`--proxy-mode`

enables the use of `X-Forwarded-*` headers through [Werkzeug's proxy support](#).

It ignores all `X-Forwarded-*` headers in case `X-Forwarded-Host` is missing from the request.

X-Forwarded-Proto and X-Forwarded-Host are used to update the request root URL, which in turn is used to update the `web.base.url` system parameter upon a successful admin authentication. This system parameter is used to generate all links for the current database; see [Web base URL of a database](#).

Warning

proxy mode *must not* be enabled outside of a reverse proxy scenario

--x-sendfile

delegates serving attachments files to the static web server and sets both X-Sendfile (apache) and X-Accel-* (nginx) http headers on stream responses. See [Serving static files and attachments](#) for web server configuration.

Logging

By default, Odoo displays all logging of [level](#) INFO, WARNING and ERROR. All logs independently of the level are output on `stderr`. Various options are available to redirect logging to other destinations and to customize the verbosity.

--logfile <file>

sends logging output to the specified file instead of `stderr`. On Unix, the file [can be managed by external log rotation programs](#) and will automatically be reopened when replaced

--syslog

logs to the system's event logger: [syslog on unices](#) and [the Event Log on Windows](#).

Neither is configurable

--log-db <dbname>

logs to the `ir.logging` model (`ir_logging` table) of the specified database. The database can be the name of a database in the "current" PostgreSQL, or [a PostgreSQL URI](#) for e.g. log aggregation.

--log-handler <handler-spec>

`{LOGGER}:{LEVEL}`, enables `LOGGER` at the provided `LEVEL` e.g. `odoo.models:DEBUG` will enable all logging messages at or above `DEBUG` level in the models.

- The colon `:` is mandatory
- The logger can be omitted to configure the root (default) handler

```
$ odoo-bin --log-handler :DEBUG --log-handler werkzeug:CRITICAL --log-handler odoo.
```

--log-web

enables DEBUG logging of HTTP requests and responses, equivalent to

`--log-handler=odoo.http:DEBUG`

--log-sql

enables DEBUG logging of SQL querying, equivalent to `--log-handler=odoo.sql_db:DEBUG`

--log-level <level>

Shortcut to more easily set predefined levels on specific loggers. "real" levels (`critical` , `error` , `warn` , `debug`) are set on the `odoo` and `werkzeug` loggers (except for `debug` which is only set on `odoo`).

Odoo also provides debugging pseudo-levels which apply to different sets of loggers:

debug_sql

sets the SQL logger to `debug`

equivalent to `--log-sql`

debug_rpc

sets the `odoo` and HTTP request loggers to `debug`

equivalent to `--log-level debug --log-request`

debug_rpc_answer

sets the `odoo` and HTTP request and response loggers to `debug`

equivalent to `--log-level debug --log-request --log-response`

Note

In case of conflict between `--log-level` and `--log-handler`, the latter is used

Multiprocessing

--workers <count>

Note

multiprocessing mode is only available on Unix-based systems

A number of options allow limiting and recycling workers:

--limit-request <limit>

Number of requests a worker will process before being recycled and restarted.

Defaults to *8196*.

--limit-memory-soft <limit>

Maximum allowed virtual memory per worker in bytes. If the limit is exceeded, the worker is killed and recycled at the end of the current request.

Defaults to *2048MiB (2048*1024*1024B)*.

--limit-memory-hard <limit>

Hard limit on virtual memory in bytes, any worker exceeding the limit will be immediately killed without waiting for the end of the current request processing.

Defaults to *2560MiB (2560*1024*1024B)*.

--limit-time-cpu <limit>

Prevents the worker from using more than <limit> CPU seconds for each request. If the limit is exceeded, the worker is killed.

Defaults to *60*.

--limit-time-real <limit>

Prevents the worker from taking longer than <limit> seconds to process a request. If the limit is exceeded, the worker is killed.

Differs from **--limit-time-cpu** in that this is a “wall time” limit including e.g. SQL queries.

Defaults to *120*.

--max-cron-threads <count>

number of workers dedicated to **cron** jobs. Defaults to 2. The workers are threads in multi-threading mode and processes in multi-processing mode.

For multi-processing mode, this is in addition to the HTTP worker processes.

Defaults to `0`.

Configuration file

Most of the command-line options can also be specified via a configuration file. Most of the time, they use similar names with the prefix `-` removed and other `-` are replaced by `_` e.g. `--db-template` becomes `db_template`.

Some conversions don't match the pattern:

- `--db-filter` becomes `dbfilter`
- `--no-http` corresponds to the `http_enable` boolean
- logging presets (all options starting with `--log-` except for `--log-handler` and `--log-db`) just add content to `log_handler`, use that directly in the configuration file
- `--smtp` is stored as `smtp_server`
- `--database` is stored as `db_name`

The default configuration file is `$HOME/.odoorc` which can be overridden using `--config`. Specifying `--save` will save the current configuration state back to that file. The configuration items relative to the command-line are to be specified in the section `[options]`.

Here is a sample file:

```
[options]
db_user=odoo
dbfilter=odoo
```

shell - Open a Shell

The Odoo command line also allows launching Odoo as a Python console environment, enabling direct interaction with the `orm` and its functionalities. Since running a shell involves starting the server, the configuration file options do apply.

```
$ odoo-bin shell
```

```
In [1]: records = env["res.partner"].search([])

In [2]: records
Out[2]: res.partner(14, 26, 33, 21, 10)

In [3]: for partner in records:
...:     partner.name = "%s !" % partner.name
...:

In [4]: env.cr.commit()
```

Important

By default, the shell is running in transaction mode. This means that any change made to the database is rolled back when exiting the shell. To commit changes, use `env.cr.commit()`.

--shell-file <init_script.py>

Specify a Python script to be run after the start of the shell. Overrides the environment variable `PYTHONSTARTUP`.

--shell-interface (ipython|ptpython|bpython|python)

Specify a preferred REPL to use in shell mode. This shell is started with the `env` variable already initialized to be able to access the `ORM` and other Odoo modules.

See also

Environment

db - Manage a Database

This command lets you manage databases through a command-line interface. The operations are specified using subcommands.

For all subcommands, these options to configure your environment are available:

- **--addons-path**

- `--db_password`
- `--db_host`
- `--db_port`
- `--db_sslmode`
- `--pg_path`

db init - Initialize a Database

This command creates a new database and installs the `base` module. You can specify the language and country of the main company.

```
$ odoo-bin db init <database>
```

database

Name of the database to be initialized.

--with-demo

Install demo data in the initialized database.

--force

Delete the database if it already exists, before initializing the new one.

--country <country_iso_code>

Code of the country to be set on the main company

--language <language_code>

Default language for the instance, default `en_US`

--username <password>

Username for the new database, default `admin`

--password <password>

Password for the new database, default `admin`

db dump - Save a Database Dump

Creates a dump file.

```
$ odoo-bin db dump <database> <dump_path>
```

(Optional) Database is dumped to specified path. By default it is dumped to `stdout`.

--format <zip | dump>

If provided, database is dumped using the specified format. Supported formats are `zip` (default), `dump` (`pg_dump` format).

--no-filestore

If provided, `zip` database is dumped without filestore

db load - Load a Database Dump

Loads a dump file into an Odoo database, the dump file can be a URL.

```
$ odoo-bin db load <database> <dump_file>
```

database

(Optional) Name of the database to create from the dump. If not provided, the dump filename without extension is used.

dump_file

`.zip` or `pg_dump` file to be loaded.

-f, --force

Delete the database if it already exists, before loading the new one.

-n, --neutralize

Neutralize the database after restoring it.

db duplicate - Duplicate a Database

Duplicate a database including filestore.

```
$ odoo-bin db duplicate <source> <target>
```

source

Name of the source database.

target

-f, --force

Delete the target database if it already exists, before initializing the new one.

db rename - Rename a Database

Rename a database from an old name to a new one.

```
$ odoo-bin db rename <source> <target>
```

source

Current name of the database.

target

New name for the database.

-f, --force

Delete the target database if it already exists, before renaming the source one.

db drop - Delete a Database

```
$ odoo-bin db drop <database>
```

database

Name of the database to drop.

i18n - Internationalization

This command has subcommands that enable you to import or export internationalization files and setup languages on the Odoo instance.

For all subcommands, these options to configure your environment are available:

- **--addons-path**
- **--config**
- **--database**

To list available codes, you can search them querying the database:

```
$ psql -d <dbname> -c "SELECT iso_code FROM res_lang ORDER BY iso_code"
```

See also

[GNU libc Locale Names](#)

Example

```
$ odoo-bin i18n loadlang -l en           # English (U.S.)  
$ odoo-bin i18n loadlang -l es es_AR     # Spanish (Spain, Argentina)  
$ odoo-bin i18n loadlang -l sr@latin     # Serbian (Latin)
```

i18n import - Import i18n files

This command imports provided translation files in the `.po` or `.csv` formats. By default, only new entries will be added, but you can choose to overwrite existing terms. All the files must refer to the specified language.

```
$ odoo-bin i18n import <files> --overwrite --language <language_code>
```

files

List of files to be imported.

Allowed extensions: `.po` , `.csv` .

-l, --language

(Required) Language code of the translations in the files.

-w, --overwrite

Overwrite existing translation when importing.

i18n export - Export i18n files

one language can be selected, and all the output will refer to that language. The `.tgz` output format makes all output be archived in a single file.

```
$ odoo-bin i18n export <modules> --languages <language_codes>
```

modules

List of modules you want to export the internationalization files from.

-l, --languages <languages>

List of language codes you want to export, `pot` for template (default).

-o, --output

Path of an only output file with translations from all provided modules.

Allowed extensions: `.po` , `.pot` , `.tgz` , `.csv`

If `-` is provided, the content is written as a `.po` file to stdout.

Only one language is allowed when this option is active.

i18n loadlang - Load language

This command loads one of the available languages into the Odoo database and activates it.

```
$ odoo-bin i18n loadlang <languages>
```

languages

language codes of the languages to be installed.

module - Manage modules

This command has subcommands that enable you to install, uninstall and upgrade modules on the Odoo instance. There's also a command to force install the demo data into the database.

For all subcommands, these options are available:

- **--addons-path**
- **--config**
- **--database**

Before installing modules, the Odoo database needs to be created and initialized on your PostgreSQL instance, e.g. using the [db init - Initialize a Database](#) command.

```
$ odoo-bin module install <modules>
```

modules

List of modules you want to install.

module uninstall - Uninstall modules

This command uninstalls all selected modules straight away.

```
$ odoo-bin module uninstall <modules>
```

modules

List of modules you want to uninstall.

module upgrade - Upgrade modules

This command upgrades all selected modules straight away.

```
$ odoo-bin module upgrade <modules>
```

modules

List of modules you want to upgrade. Use `base` or `all` for all installed modules.

--outdated

Only update modules that have a newer version on disk. If `all` is used as `modules` argument, this applies to all installed modules.

module forcedemo - Force demo data install

This command forces the installation of [Demo Data](#)

Warning

There are no additional options to this command.

neutralize - Neutralize a Database

The Odoo command line allows for neutralizing a database as well. The command must be run with a database option.

```
$ odoo-bin --addons-path <PATH,...> neutralize -d <database>
```

-d <database> , --database <database>

Specify the database name that you would like to neutralize.

--stdout

Output the neutralization SQL instead of applying it

See also

[Neutralized database](#)

scaffold - Scaffold a Module

Scaffolding is the automated creation of a skeleton structure to simplify bootstrapping (of new modules, in the case of Odoo). While not necessary it avoids the tedium of setting up basic structures and looking up what all starting requirements are.

Scaffolding is available via the `odoo-bin scaffold` subcommand.

```
$ odoo-bin scaffold my_module /addons/
```

name (required)

the name of the module to create, may munged in various manners to generate programmatic names (e.g. module directory name, model names, ...)

destination (default=current directory)

directory in which to create the new module, defaults to the current directory

This will create module *my_module* in directory */addons/*.

populate - Populate a Database

Odoo Populate allows to duplicate existing data in a given database. This can be used for testing and benchmarking when large tables are needed. The duplication process introduces variation for some fields to respect `UNIQUE` constraints, among other things. It also follows x2Many relationships.

```
$ odoo-bin populate -d my_database --models res.partner,account.move --factors 100
```

-d <database>

name of the database to populate

--models <models>

list of models to populate. Models appearing twice will only be populated once.

--factors <factors>

list of populate factors. In case a factor is missing for a model, the last factor in the list will be used.

--sep <separator>

separator used to generate record names

cloc - Count Lines of Code

Odoo Cloc is a tool to count the number of relevant lines of code written in Python, Javascript, CSS, SCSS, or XML. This can be used as a rough metric for pricing maintenance of extra modules.

```
$ odoo-bin cloc -c config.conf -d my_database
```

-d <database> , --database <database>

Process the code of all extra modules installed on the provided database, and of all server actions and computed fields manually created in the provided database.

process.

```
$ odoo-bin cloc --addons-path=addons -d my_database
```

See also

- [With the --database option](#)

-p <path> , --path <path>

Process the files in the provided path.

If combined with **--database**, the count will be that of the sum of both options' results (with possible overlaps). At least one of these two options is required to specify which code to process.

```
$ odoo-bin cloc -p addons/account
```

Multiple paths can be provided by repeating the option.

```
$ odoo-bin cloc -p addons/account -p addons/sale
```

See also

- [With the --path option](#)

--addons-path <directories>

Comma-separated list of directories in which modules are stored. These directories are scanned for modules.

Required if the **--database** option is used.

-c <directories>

Specify a configuration file to use in place of the **--addons-path** option.

-v , --verbose

With the `--database` option

Odoo Cloc counts the lines in each file of extra installed modules in a given database. In addition, it counts the Python lines of server actions and custom computed fields that have been directly created in the database or imported. Finally, it counts the lines of code of Javascript, CSS, and SCSS files, and of QWeb views from imported modules.

Some files are excluded from the count by default:

- The manifest (`__manifest__.py` or `__openerp__.py`)
- The contents of the folder `static/lib`
- The tests defined in the folder `tests` and `static/tests`
- The migrations scripts defined in the folder `migrations` and upgrades
- The XML files declared in the `demo` or `demo_xml` sections of the manifest

For special cases, a list of files that should be ignored by Odoo Cloc can be defined per module. This is specified by the `cloc_exclude` entry of the manifest:

```
"cloc_exclude": [  
    "lib/common.py", # exclude a single file  
    "data/*.xml",    # exclude all XML files in a specific folder  
    "example/**/*.py", # exclude all files in a folder hierarchy recursively  
    "**/*.scss",       # exclude all scss file from the module  
]
```

The pattern `**/*` can be used to ignore an entire module. This can be useful to exclude a module from maintenance service costs.

For more information about the pattern syntax, see [glob](#).

With the `--path` option

This method works the same as with the `--database option` if a manifest file is present in the given folder. Otherwise, it counts all files.

Identifying Extra Modules

To distinguish between standard and extra modules, Odoo Cloc uses the following heuristic: modules that are located (real file system path, after following symbolic links) in the same

Error Handling

Some file cannot be counted by Odoo Cloc. Those file are reported at the end of the output.

Max file size exceeded

Odoo Cloc rejects any file larger than 25MB. Usually, source files are smaller than 1 MB. If a file is rejected, it may be:

- A generated XML file that contains lots of data. It should be excluded in the manifest.
- A JavaScript library that should be placed in the `static/lib` folder.

Syntax Error

Odoo Cloc cannot count the lines of code of a Python file with a syntax problem. If an extra module contains such files, they should be fixed to allow the module to load. If the module works despite the presence of those files, they are probably not loaded and should therefore be removed from the module, or at least excluded in the manifest via `cloc_exclude`.

obfuscate - Obfuscate database

This command provides a quick and easy way to obfuscate some of the data in the Odoo instance, mainly used for instructional purposes or to make quick videos for the support team helping technicians avoid leaking sensitive information.

Warning

This command must be used carefully, as it is **not** considered a safe method for full anonymizing data before transfer to a third party. Images, PDF attachments, amounts, many other informations may not be obfuscated and cause sensitive information leaks. A thorough review is required before sharing data to ensure that no sensitive information is exposed.

Obfuscation is symmetric, so content can be unobfuscated using the same password.

All the configurations available for the `server` command are available here too.

```
$ odoo-bin obfuscate --pwd=<password>
```


if you want to unobfuscate instead of obfuscate.

--fields <fields>

comma-separated list of `table.column` entries to obfuscate/unobfuscate.

--file <file>

file containing the list of `table.column` entries to obfuscate/unobfuscate.

--exclude

comma-separated list of `table.column` entries not to obfuscate/unobfuscate.

--allfields

used only when **--unobfuscate** is selected.

Try to unobfuscate all fields. It's slower than specifying the fields manually.

--vacuum

used only when **--unobfuscate** is selected.

After unobfuscation, completely clear the obfuscated tables and reclaim unused disk space.

--pertablecommit

commit once per table, after obfuscation.

It avoids big transactions that might get a timeout or face rollback after an error.

-y, --yes

don't ask for manual confirmation.

Only use if you're sure that you're not going to leak sensible information by sharing the database to third party without a review.

deploy - Deploy module remotely

This command uploads a module to a remote Odoo server and installs it. It's simpler than manually connecting to the remote server, and it does not require full access to the machine that hosts the Odoo instance, only the Odoo administrative credentials.

```
$ odoo-bin deploy <path> <url> --db <dbname> --login <login> --password <password>
```

- The server must have the module `base_import_module` installed.
- The user selected with the `--login` option must have administrative rights.

path

path of the module to be deployed

url

(Optional) url of the server where the module must be deployed (default `http://localhost:8069`)

db <dbname>

database name (if the server does not use the `--db-filter` option)

--login <username>

name of the user with admin rights (default `admin`)

--password <password>

password of the user with admin rights (default `admin`)

--verify-ssl

verify the server's SSL certificate, to ensure the target instance is legit.

--force

re-initialize the module in case it's already installed. It will update `noupdate="1"` records.

upgrade_code - Rewrite source code

This command rewrites the entire source code using the scripts found in the `odoo/upgrade_code` folder. It is used for doing the heavy-lifting when dealing with big code migrations and forward ports.

Note

All the scripts are doing a best-effort at migrating the source code, but they are not silver bullets.

It accepts the `--addons-path` option.

--script <path>

run a single script

--from <version>

run all script starting from this version included

--to <version>

to be used with **--from**. Run all scripts up to this version included. Default is `odoo.release.version`.

--glob <glob>

filter the files to be rewritten, default is every file (`**/*`)

--dry-run

list the files that would be re-written, but do not apply changes

Code upgrade scripts

The scripts must be named `{version}-{name}.py`, and must expose an `upgrade` function that takes a single `file_manager` argument and have no return.

The `file_manager` argument is a sequence of `files`, which have 3 attributes and some helper methods:

- `path`: the `pathlib.Path` where the file is on the file system.
- `addon`: the addon the file belongs to.
- `content`: the re-writable content of the file (lazy).
- `print_progress(current, total)`: outputs the current progress.

Example

```
def upgrade(file_manager):
    files = (file for file in file_manager if file.path.suffix == '.py')
    for fileno, file in enumerate(files, start=1):
        file.content = file.content.replace(..., ...)
        file_manager.print_progress(fileno, len(files))
```



[Contact Support](#)

[Ask the Odoo Community](#)

